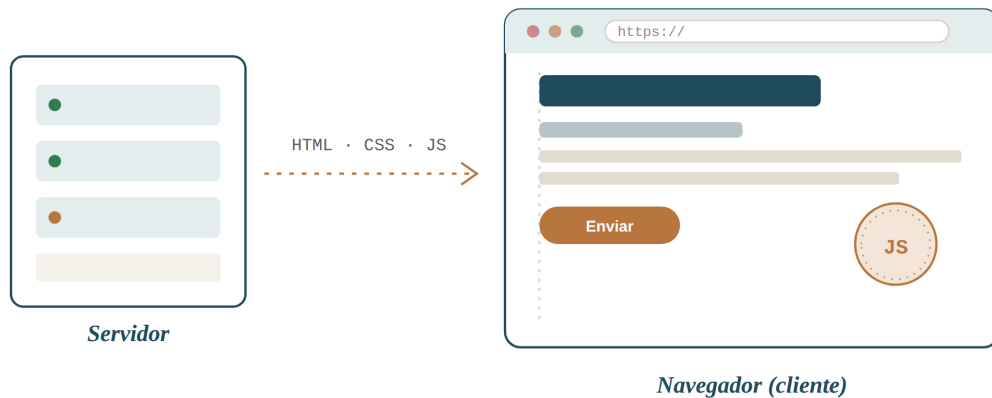


Tema 1

Introducción a la programación en el cliente web



Antes de escribir la primera línea de JavaScript, conviene entender dónde y cómo se ejecuta ese código: qué diferencia una aplicación de servidor de una aplicación de cliente, qué es realmente el navegador por dentro, y con qué herramientas se trabaja hoy en el desarrollo profesional.

En este tema

1. Modelos de ejecución de código: servidor vs. cliente
2. El navegador como entorno de ejecución
3. Lenguajes de programación en entorno cliente
4. Programación de guiones vs. programación tradicional
5. Integración del código JavaScript con HTML
6. Herramientas y entornos de desarrollo

1.1 Modelos de ejecución de código: servidor vs. Cliente

Cuando un usuario visita una página web, el código que hace que esa página "funcione" puede ejecutarse en dos lugares distintos: en el **servidor** (antes de que la página llegue al navegador) o en el **cliente** (dentro del propio navegador del usuario, una vez recibida la página). Entender esta diferencia es la base de todo lo que se verá en este módulo.

Ejecución en el servidor (server-side)

El servidor recibe una petición HTTP, ejecuta código (PHP, Node.js, Python, Java...), genera una respuesta (normalmente HTML ya construido) y la envía al navegador. El navegador se limita a mostrar lo que recibe.

```
Navegador → Petición HTTP (GET /productos) → Servidor
Servidor → Ejecuta código, consulta BD, genera HTML → Navegador
Navegador → Renderiza el HTML recibido
```

Características:

- El código fuente nunca llega al cliente, el usuario solo ve el HTML resultante.
- Cada interacción que requiera datos nuevos implica, en su forma más básica, una recarga completa de la página.
- El servidor tiene acceso directo a la base de datos, ficheros del sistema, variables de entorno, etc.

Ejecución en el cliente (client-side)

El navegador recibe HTML, CSS y JavaScript, y es el propio navegador quien ejecuta ese JavaScript, ya con la página cargada. El código se ejecuta en la máquina del usuario, no en la del servidor.

```
Navegador → Petición HTTP (GET /productos) → Servidor
Servidor → Envía HTML + CSS + JS (sin "resolver" nada) → Navegador
Navegador → Ejecuta el JavaScript recibido, modifica la página en tiempo real
```

Características:

- El código fuente sí llega al cliente: cualquier usuario puede abrir las herramientas de desarrollador y leerlo (e incluso modificarlo en su propia sesión).
- Permite interactividad inmediata sin recargar la página (validar un formulario, desplegar un menú, actualizar una parte de la pantalla).
- No tiene acceso directo a la base de datos ni al sistema de ficheros del servidor; si necesita esos datos, debe pedirlos mediante una comunicación asíncrona.

Comparativa

Aspecto	Servidor (server-side)	Cliente (client-side)
¿Dónde se ejecuta?	En la máquina del servidor	En el navegador del usuario
¿Quién ve el código fuente?	Nadie (solo el resultado)	Cualquier usuario
Acceso a BD / ficheros	Directo	Ninguno (requiere API)

Interactividad sin recarga	No (por sí solo)	Sí
Ejemplos de lenguajes	PHP, Node.js, Python	JavaScript
Depende del navegador del usuario	No	Sí

Nota: No son modelos excluyentes: la inmensa mayoría de aplicaciones reales combinan ambos. El servidor genera y sirve los datos (a través de una API), y el cliente los consume y los presenta de forma interactiva. El backend expone una API REST y Javascript, en el navegador, la consume y construye la interfaz.

1.2 El navegador como entorno de ejecución

El navegador no es solo un lector de HTML, es una plataforma de ejecución de código con su propia arquitectura interna.

Motor de renderizado y motor de JavaScript

Cada navegador incorpora dos piezas fundamentales:

- **Motor de renderizado (rendering engine):** interpreta HTML y CSS y construye lo que se ve en pantalla (Blink en Chrome/Edge, Gecko en Firefox, WebKit en Safari).
- **Motor de JavaScript (JS engine):** interpreta y ejecuta el código JavaScript (V8 en Chrome/Edge, SpiderMonkey en Firefox, JavaScriptCore en Safari).

Ambos motores colaboran: el JavaScript puede leer y modificar lo que el motor de renderizado ha construido (el DOM), y el motor de renderizado vuelve a pintar la pantalla cuando ese contenido cambia.

ECMAScript vs. APIs del navegador

Este es uno de los puntos que más confusión genera al alumnado, así que conviene remarcarlo desde el principio.

- ECMAScript es la especificación del lenguaje JavaScript en sí: variables, tipos de datos, funciones, clases, bucles, Array, Math, Promise... Es lo mismo estés donde estés (navegador o Node.js).
- Las APIs del navegador (Web APIs) son funcionalidades que el navegador añade por encima del lenguaje: document, window, fetch, localStorage, eventos del ratón... No forman parte de JavaScript como lenguaje; las proporciona el entorno de ejecución.

```
// Esto es JavaScript puro (ECMAScript), funciona igual en el navegador y en Node.js
const precios = [19.99, 45.5, 12.0];
const total = precios.reduce((acumulado, precio) => acumulado + precio, 0);

// Esto es una Web API del navegador – NO existe en Node.js
document.querySelector('#total').textContent = `${total.toFixed(2)} €`;
```

Importante: Si intentas ejecutar código que use `document` o `window` en un entorno que no sea un navegador (por ejemplo, en Node.js), obtendrás un error como `document is not defined`. Esto es porque `document` no es JavaScript: es una API que solo existe cuando el código se ejecuta dentro de un navegador.

El sandbox de seguridad

El código JavaScript de una página web se ejecuta dentro de un entorno restringido (sandbox): no puede leer ni escribir ficheros del ordenador del usuario, no puede acceder a otras pestañas de dominios distintos (salvo mecanismos controlados) y no puede ejecutar comandos del sistema operativo. Esta restricción es deliberada: si una página web pudiera hacer cualquier cosa en tu ordenador, visitar cualquier web sería un riesgo de seguridad inasumible.

1.3 Lenguajes de programación en entorno cliente

JavaScript: el lenguaje nativo del navegador

JavaScript es, hoy en día, el único lenguaje que todos los navegadores ejecutan de forma nativa, sin necesidad de instalar nada adicional. Por eso, aunque existan alternativas, todas terminan convirtiéndose en JavaScript antes de llegar al navegador.

TypeScript

TypeScript es un superconjunto de JavaScript que añade tipado estático. El navegador no entiende TypeScript: el código se transpila (se convierte) a JavaScript antes de ejecutarse, normalmente mediante un proceso de compilación en el entorno de desarrollo.

```
// TypeScript – el navegador NO puede ejecutar esto directamente
function calcularTotal(precios: number[]): number {
  return precios.reduce((acc, precio) => acc + precio, 0);
}

// JavaScript resultante tras la transpilación – esto sí lo ejecuta el
navegador
function calcularTotal(precios) {
  return precios.reduce((acc, precio) => acc + precio, 0);
}
```

Tecnologías históricas ya obsoletas

Antes de que JavaScript se consolidara como estándar único, existieron alternativas hoy completamente abandonadas: Java Applets, Adobe Flash (ActionScript) y VBScript. Todas requerían un plugin instalado en el navegador, lo cual generaba problemas de seguridad y de compatibilidad. Los navegadores modernos ya no las soportan.

Nota: Es útil entender por qué JavaScript "ganó": no fue por ser técnicamente superior en 1995, sino porque es el único lenguaje que corre de forma nativa en cualquier navegador sin instalar nada. Esa característica (cero fricciones para el usuario final) ha sido decisiva en toda la historia de la web.

1.4 Programación de guiones vs. programación tradicional

JavaScript nació como un lenguaje de guiones (scripting language), pensado para añadir pequeñas dosis de interactividad a documentos HTML, no para construir aplicaciones completas. Aunque hoy se usa para aplicaciones muy complejas, conserva rasgos propios de esa naturaleza que lo diferencian de lenguajes "tradicionales" compilados como C o Java.

Característica	Lenguaje de guiones (JS)	Lenguaje compilado (Java, C)
Compilación previa	No (interpretado / JIT)	Sí, obligatoria antes de ejecutar
Tipado	Dinámico (débil)	Estático (fuerte)
Ciclo de desarrollo	Editar → recargar → ver resultado	Editar → compilar → ejecutar
Acoplamiento al entorno	Depende del navegador	Independiente del SO/entorno
Detección de errores	En tiempo de ejecución	En tiempo de compilación

Ventajas de la programación de guiones en el cliente

- **Feedback inmediato:** no hay que recompilar ni desplegar nada para ver un cambio.
- **Interactividad sin recargar la página:** validar un formulario, mostrar/ocultar contenido, animar elementos.
- **Reduce la carga del servidor:** tareas como validar un campo de texto no necesitan ida y vuelta al servidor.
- **Accesible:** cualquier navegador con un editor de texto es suficiente para empezar.

Desventajas

- **El código es visible y modificable** por el usuario final.
- **Los errores se detectan tarde**, en tiempo de ejecución, no al "compilar".
- **Depende del navegador del usuario:** distintos navegadores pueden implementar ciertas APIs de forma distinta.
- **Rendimiento limitado** para cálculos muy pesados, comparado con lenguajes compilados a código nativo.

Importante: Como el código cliente es visible y manipulable por el usuario, nunca debe considerarse una capa de seguridad. Validar un formulario en el cliente mejora la experiencia de usuario (feedback inmediato), pero un atacante puede saltarse esa validación fácilmente. La validación real y definitiva de cualquier dato siempre debe repetirse en el servidor. La validación en el cliente es una ayuda, no una garantía.

1.5 Integración del código JavaScript con HTML

Existen varias formas de incorporar JavaScript a un documento HTML. Elegir la correcta no es solo una cuestión de estilo: afecta al rendimiento de carga de la página y a la organización del proyecto.

Código en línea (inline)

El código se escribe directamente como valor de un atributo HTML (onclick, onchange...).

```
<!-- Evitar: mezcla contenido (HTML) con comportamiento (JS) -->
<button onclick="alert('Producto añadido')">Añadir al carrito</button>
```

Importante: El código inline está considerado mala práctica en desarrollo profesional. Mezcla estructura (HTML) y comportamiento (JavaScript) en el mismo sitio, dificulta el mantenimiento, complica la depuración y no funciona si se aplica una política de seguridad de contenido (CSP) estricta, algo habitual en aplicaciones reales. Se muestra aquí únicamente para reconocerlo en código legado, no para utilizarlo.

Código interno (*internal*)

El código se escribe dentro de una etiqueta `<script>` en el propio documento HTML. Útil para pruebas rápidas, pero no recomendado en proyectos reales: no se puede reutilizar en otras páginas ni cachear de forma independiente.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Panel de administración</title>
</head>
<body>
  <h1 id="bienvenida"></h1>

  <script>
    document.querySelector('#bienvenida').textContent = 'Bienvenido al panel';
  </script>
</body>
</html>
```

Código externo (recomendado)

El código se escribe en un fichero `.js` independiente, enlazado mediante el atributo `src`.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Panel de administración</title>
  <script src="./js/panel.js" defer></script>
</head>
<body>
  <h1 id="bienvenida"></h1>

</body>
</html>
```

El fichero `panel.js`:

```
document.querySelector('#bienvenida').textContent = 'Bienvenido al panel';
```

Ventajas de la separación en ficheros externos:

- El fichero .js puede reutilizarse en varias páginas.
- El navegador puede **cachearlo**, evitando descargarlo de nuevo en visitas posteriores.
- Facilita el trabajo en equipo (HTML y JS se pueden editar por separado sin conflictos).
- Permite aplicar herramientas de calidad de código (linters, formateadores) sobre ficheros .js de forma independiente.

async vs. defer: cuándo se ejecuta el script

Cuando el navegador encuentra una etiqueta <script>, por defecto **detiene el parseo del HTML** hasta descargar y ejecutar ese script. Los atributos async y defer cambian este comportamiento.

Atributo	Descarga	Ejecución	Orden garantizado
(sin atributo)	Bloquea el parseo	Inmediata, bloqueando	Sí
Async	En paralelo, no bloquea	En cuanto termina de descargar	No
Defer	En paralelo, no bloquea	Al terminar de parsear el HTML	Sí

Nota: Como regla práctica: usa defer para el código de tu aplicación (necesitas que el DOM esté completo y que el orden entre ficheros se respete), y async para scripts independientes que no interactúan con el DOM ni con otros scripts.

type="module"

Cuando el código se organiza en módulos ES, se indica con type="module". Los módulos se comportan como si llevaran defer de forma automática. Profundizaremos en el sistema de módulos más adelante.

1.6 Herramientas y entornos de desarrollo

Herramientas de desarrollador del navegador (*DevTools*, botón derecho + *Inspeccionar*)

- **Console:** ejecuta JavaScript de forma interactiva y muestra mensajes (console.log, console.error, console.table...).
- **Elements/Inspector:** muestra el DOM en tiempo real y permite modificarlo para probar cambios sin tocar el fichero fuente.
- **Sources/Debugger:** permite colocar *breakpoints*, ejecutar el código paso a paso e inspeccionar variables en cada momento.
- **Network:** muestra todas las peticiones realizadas por la página (fundamental en, comunicación asíncrona).

```
function calcularDescuento(precio, porcentaje) {  
  debugger; // pausa la ejecución si las DevTools están abiertas  
  return precio - (precio * porcentaje) / 100;  
}
```

Nota: console.log es útil, pero abusar de él para depurar es poco eficiente en proyectos grandes. Acostumbrarse a usar breakpoints en el panel Sources desde el principio ahorra mucho tiempo más adelante.

Editor de código

Visual Studio Code es, hoy en día, el estándar de facto en desarrollo web. Algunas extensiones prácticamente imprescindibles:

- **ESLint:** detecta errores y malas prácticas mientras escribes.
- **Prettier:** formatea el código de forma automática y consistente.
- **Live Server:** levanta un servidor local con recarga automática al guardar cambios.
- **Indent Rainbow:** solo colorea visualmente los niveles de sangría para que sea más fácil ver bloques anidados. No modifica el código ni impone estilo.

Actividades prácticas

1. Instalar VS Code y añadir las extensiones Live Server, indent-rainbow y Prettier (lee la documentación de la extensión para activarlo). Crear un proyecto con un archivo index.html, y probar que los cambios se recargan automáticamente en el navegador.

2. Usar DevTools en Chrome/Firefox: abrir una página web conocida (por ejemplo, Wikipedia), inspeccionar su estructura, modificar un estilo y ver el resultado en vivo.